



Diseño de software

Clase asincrónica #10

Elaborado por:

Isaac Elizondo Espinoza

Profesor: Luis Javier Chavarría Sánchez

I Semestre, 2026

1. Porque el Observer necesita mantener una referencia al Subject para poder consultar su estado cuando reciba una notificación. De esta manera puede reaccionar a los cambios ocurridos en el objeto observado.
2. Porque un Subject puede no tener observadores registrados o puede tener múltiples observadores interesados en los cambios de estado. El patrón permite notificar a cualquier cantidad de observadores.
3. Sí. Un mismo Subject puede tener múltiples observadores concretos que reaccionen de forma distinta ante un mismo evento. Por ejemplo, uno puede generar gráficos, otro enviar alertas y otro almacenar registros históricos.
4. Porque el observador necesita conocer cuál Subject va a observar. Además, durante su construcción puede registrarse automáticamente en dicho Subject para comenzar a recibir notificaciones cuando ocurra un cambio de estado.
5.
 - a. En términos generales sí. El Subject tiene la responsabilidad de gestionar cambios de estado y notificaciones, mientras que los Observers se encargan de reaccionar ante esos cambios. Además, es posible agregar nuevos observadores sin modificar el Subject, lo cual favorece el Principio Abierto/Cerrado (OCP). También se promueve el desacoplamiento entre el emisor de eventos y quienes reaccionan a ellos.
 - b. Sí, existen algunos posibles problemas. El Subject depende directamente de la clase TemperaturaObserver en lugar de depender de una abstracción más general. Además, la lógica de presentación aparece dentro del método update() mediante System.out.println(), mezclando responsabilidades. Estas decisiones aumentan el acoplamiento y dificultan la reutilización del código.
6. Los patrones creacionales se enfocan en la creación e inicialización de objetos, mientras que los patrones de comportamiento se enfocan en cómo los objetos colaboran e intercambian responsabilidades durante la ejecución del sistema.
7. Porque su propósito principal es definir la comunicación entre objetos cuando ocurre un cambio de estado. No se enfoca en crear objetos ni en modificar estructuras, sino en coordinar comportamientos y notificaciones entre entidades.
8. El patrón Observador tiene como ventaja que reduce el acoplamiento entre el Subject y los Observers, permite agregar nuevos observadores sin modificar el Subject, facilita la extensión del sistema y favorece la reutilización de componentes. Sin embargo, también puede dificultar el seguimiento del flujo de ejecución cuando existen muchos observadores, puede afectar el rendimiento si se generan demasiadas notificaciones y puede complicar la depuración cuando participan muchas entidades observadoras.
9. Sí. InterfaceOriginal podría implementarse como una superclase abstracta en lugar de una interfaz, siempre que proporcione el comportamiento común requerido por las clases concretas y decoradores. Sin embargo, utilizar una interfaz ofrece mayor flexibilidad y menor acoplamiento.
10. Porque el decorador necesita mantener una referencia al objeto que está decorando. Gracias a esta relación puede ejecutar la funcionalidad original y posteriormente agregar comportamiento adicional sin modificar el objeto base.
11. Porque representa un detalle interno de implementación del decorador. No forma parte de la interfaz pública del objeto decorado y únicamente debe ser utilizado por la propia clase para complementar la funcionalidad original.
12. Una decoración horizontal atiende requerimientos donde se agregan funcionalidades alternativas o complementarias a un mismo nivel, sin necesidad de construir una cadena profunda de decoraciones.
13. Una decoración vertical atiende requerimientos donde varias funcionalidades deben acumularse progresivamente sobre un mismo objeto, formando una cadena de decoradores.

14. La nueva decoración debe tener relación estructural con Decorador, no directamente con Decoracion1. Esto permite que cualquier decorador pueda envolver tanto al objeto original como a otro decorador, manteniendo la flexibilidad y extensibilidad del diseño.

15.

a. Sí. El patrón Decorador cumple especialmente con el Principio Abierto/Cerrado (OCP), ya que permite extender funcionalidades sin modificar código existente. Además, los decoradores pueden sustituir al objeto original porque mantienen la misma interfaz, lo cual respeta el Principio de Sustitución de Liskov (LSP).

b. No se observan olores importantes en la estructura base. Sin embargo, un uso excesivo de decoradores podría generar una cadena difícil de comprender y mantener, aumentando la complejidad del sistema.

16. No existe un límite fijo. Sin embargo, se recomienda mantener pocas decoraciones encadenadas para evitar que el comportamiento final sea difícil de comprender y depurar.

17. Porque modifica la estructura de colaboración entre objetos mediante el encapsulamiento de un objeto dentro de otro. No se enfoca en crear objetos ni en definir comunicación por eventos, sino en agregar responsabilidades manteniendo la estructura flexible del sistema.

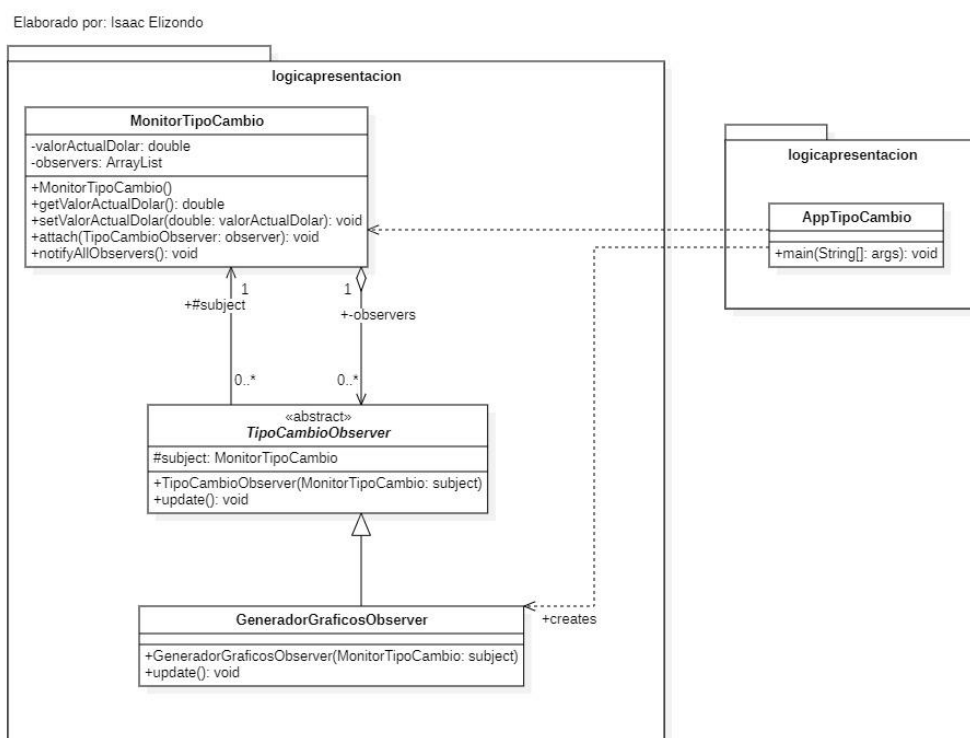
18. Se utiliza cuando se requiere agregar responsabilidades o funcionalidades adicionales a un objeto de forma dinámica, evitando modificar la implementación original y manteniendo el sistema extensible.

2.

Contexto Observador

En un sistema de control de decisiones sobre inversión, un monitor del tipo de cambio del dólar analiza variaciones importantes en su valor. Cuando detecta cambios drásticos entre picos del mercado, estos pueden representar riesgos o posibles oportunidades para los inversionistas. Por esta razón, un generador de gráficos debe actualizar automáticamente la información visual cada vez que ocurre una variación relevante.

En este contexto, el evento observado corresponde a los cambios drásticos en el valor del dólar. El monitor del tipo de cambio actúa como sujeto, ya que mantiene una lista de observadores interesados en esos cambios. El generador de gráficos actúa como observador, porque recibe la notificación y actualiza la representación visual del comportamiento del mercado para facilitar el análisis posterior.



Contexto decorador

Inicialmente, una aplicación de cálculo matemático mostraba el valor del número π (pi) con una representación básica de decimales conocidos. Sin embargo, posteriormente surgió la necesidad de agregar nuevas formas de presentación, como notación científica, formato para impresión académica y exportación del valor en un formato especializado para estudios matemáticos.

Modificar la clase original cada vez que se requiera una nueva forma de presentación comprometería la estabilidad y mantenibilidad del sistema. Por esta razón, el patrón Decorador permite envolver el objeto original y agregar nuevas funcionalidades de representación sin alterar la clase base.

